



Mangrove

Security Review

Cantina Managed review by:

M4rio.eth, Lead Security Researcher

Tnch, Security Researcher

April 3, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	High Risk	4
3.1.1	Morpho rewards can not be claimed due to missing <code>claimRewardsForToken</code> implementation	4
3.1.2	Morpho rewards cannot be claimed due to flawed integration with rewards distributor	4
3.2	Medium Risk	4
3.2.1	The token balance of the <code>ERC4626Router</code> will return wrong balances for vaults with fees	4
3.2.2	Missing slippage checks on deposits and withdrawals could result in sandwich attacks	5
3.3	Low Risk	6
3.3.1	The <code>ERC4626Router</code> does not support nested vaults	6
3.3.2	<code>ERC4626</code> vaults with hooks on deposits could reenter in the make offer flow	7
3.3.3	Rounding up in <code>ERC4626</code> withdrawal can cause the withdraw funds to fail	8
3.4	Informational	9
3.4.1	Documentation and minor issues	9
3.4.2	Missing events	9
3.4.3	Missing <code>forceApprove</code> on <code>ERC4626Router</code> deposit	10
3.4.4	Lack of validation for supported asset in <code>setVaultForToken</code> function	10
3.4.5	Low-level calls should use <code>abi.encodeCall</code> to create calldata	10
3.4.6	Various Kandel functions won't be able to be called via the <code>Man-groveERC4626KandelVault</code> contract	11

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	<i>Must fix as soon as possible (if already deployed).</i>
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Mangrove is an order book DEX that allows for trading strategies without locking funds.

From Mar 12th to Mar 15th the Cantina team conducted a review of [mangrove-monorepo](#) on commit hash [9dbdc2ac](#). The team identified a total of **13** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	2	2	0
Medium Risk	2	2	0
Low Risk	3	1	2
Gas Optimizations	0	0	0
Informational	6	4	2
Total	13	9	4

3 Findings

3.1 High Risk

3.1.1 Morpho rewards can not be claimed due to missing `claimRewardsForToken` implementation

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: The `MorphoVaultRouter` is an extension of the `ERC4626Router` designed specifically for Morpho Vaults. Morpho vault depositors not only earn interest from underlying borrowers but also accrue MORPHO token rewards, as Morpho incentivizes lending. However, since the router contract is the entity supplying funds to Morpho, it accrues all MORPHO rewards on behalf of its depositors. To claim these rewards, the admin of the `MorphoVaultRouter` must call `claimRewardsForToken`:

```
function claimRewardsForToken(IERC20 token, uint amount, bytes32[] calldata proof, address receiver)
    external
    onlyAdmin
{
    MORPHO_REWARD_DISTRIBUTOR.claim(msg.sender, address(token), amount, proof);
    token.transfer(receiver, amount);
    emit RewardsClaimed(token, amount, receiver);
}
```

When a router is deployed for a specific Kandel, the router's admin is transferred to `ERC4626Kandel`. After the rewards are collected, the Kandel must call `adminWithdrawTokens` to withdraw them.

The issue is that `ERC4626Kandel` has no way to call `claimRewardsForToken` on the router, which results in lost Morpho rewards.

Recommendation: Consider creating an `ERC4626Kandel` tailored for Morpho that can call `claimRewardsForToken`.

Mangrove: Fixed in commit [7274acd4](#).

Cantina Managed: Fix verified.

3.1.2 Morpho rewards cannot be claimed due to flawed integration with rewards distributor

Severity: High Risk

Context: (No context files were provided by the reviewer)

Description: The `claimRewardsForToken` function of the `MorphoVaultRouter` contract is intended for the admin to claim token rewards from Morpho's rewards distributor. To do this, the function first intends to call `IMorphoRewardDistributor::claim` and then transfer the received tokens to a `receiver` account passed as parameter by the admin.

However, the first argument passed to `claim` is `msg.sender` - that is, the admin's account. As a consequence, the `MorphoVaultRouter` never receives the tokens from the distributor, and the token transfer after the call to `claim` will inevitably fail. Thus breaking the whole claiming process.

Recommendation: Modify the `claimRewardsForToken` function so that it correctly integrates with Morpho's rewards distributor contract, ensuring that the `MorphoVaultRouter` contract is the one receiving the rewards before transferring them to the intended `receiver` account. Also, consider improving the integration tests to ensure this integration with Morpho's contract works as intended.

Mangrove: Fixed in commit [46cea0c3](#).

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 The token balance of the `ERC4626Router` will return wrong balances for vaults with fees

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: The `_tokenBalance` in the `ERC4626Router` is used to return the balance of a token hold by the router, this is computed by getting the balance of the router and the number of assets hold by the router in the the token's vault by calling `convertToAssets`:

```
function _tokenBalance(IERC20 token) public view returns (uint balance) {
    balance = token.balanceOf(address(this));
    IERC4626 vault = vaults[token];
    if (address(vault) != address(0)) {
        balance += vault.convertToAssets(vault.balanceOf(address(this)));
    }
}
```

The problem with this is that if the vault has fees on withdraw, the `convertToAssets` will not take that into consideration according to the EIP, this will result in untruthful balance returned by this function.

`convertToAssets`

Must NOT include any fees that are charged against assets in the vault.

Recommendation: Consider using `previewRedeem` as this should return the closes number of assets that would be received if redeem would have been called.

`previewRedeem`

MUST return as close to and no more than the exact amount of assets that would be withdrawn in a redeem call in the same transaction. I.e. redeem should return the same or more assets as `previewRedeem` if called in the same transaction.

Mangrove: Fixed in commit [3a997a9e](#).

Cantina Managed: Fix verified.

3.2.2 Missing slippage checks on deposits and withdrawals could result in sandwich attacks

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: It is a known issue that ERC4626 deposit and withdrawal operations lack slippage checks. The EIP does not enforce them, leaving it to integrators to ensure slippage checks are implemented whenever a deposit or withdrawal is performed. Every deposit or withdrawal creates an opportunity for MEV by manipulating the share price, which can result in fewer shares or assets being received by the vault. A good example where slippage can cause significant damage is the `setVaultForToken` function. The router's admin can set or change a vault for a specific token by calling this function.

The flow is as follows:

1. Suppose the router currently holds an `oldVault` for a token `T` and has "shares" of that vault.
2. When an admin calls `setVaultForToken(T, newVault)`, the code:
 1. Redeems 100% of the shares from `oldVault` into the underlying token `T`.
 2. Sets `vaults[T] = newVault`.
 3. Deposits all token `T` holdings into the newly assigned vault.

Since both `_deposit` and `redeem` lack slippage checks, they can be easily exploited through MEV. The above flow can be summarized as:

- Redeem up to `oldVault.maxRedeem(...)` shares.
- Deposit all resulting token `T` holdings into `newVault`.

If the share prices of the old and new vaults can be manipulated, an attacker could sandwich the change, causing the vault to withdraw fewer assets and deposit even less. This is possible by decreasing the share price during redemption from the old vault and increasing it during the deposit into the new vault.

This issue currently has a medium impact because the planned deployment is only for Arbitrum and Base, which do not have a public mempool, making exploitation less frequent. However, accidental MEV is still possible.

```

function setVaultForToken(IERC20 token, IERC4626 vault) public virtual onlyAdmin {
    // Verify that the token is not the zero address
    require(address(token) != address(0), "ERC4626Router/zeroToken");

    // If a previous vault exists, withdraw all assets
    IERC4626 oldVault = vaults[token];
    if (address(oldVault) != address(0)) {
        uint shares = oldVault.balanceOf(address(this));
        if (shares > 0) {
            uint maxRedeemable = oldVault.maxRedeem(address(this));
            require(maxRedeemable >= shares, "ERC4626Router/maxRedeemExceeded");
            oldVault.redeem(maxRedeemable < shares ? maxRedeemable : shares, address(this), address(this));
        }
    }

    // Set the new vault
    vaults[token] = vault;

    // Re-deposit the token
    _deposit(token);
}

```

Recommendation: Consider adding slippage checks throughout the router for both deposit and withdrawal operations from the vault.

Mangrove: Fixed in commit [2d9e736c](#).

Cantina Managed: Fix verified.

3.3 Low Risk

3.3.1 The ERC4626Router does not support nested vaults

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: The ERC4626Router deposits funds into various vaults associated with different tokens. The shares of these vaults are held in the router for later redemption:

```

function _deposit(IERC20 token) internal {
    IERC4626 vault = vaults[token];
    if (address(vault) != address(0)) {
        uint balance = token.balanceOf(address(this));
        if (balance > 0) {
            uint maxDeposit = vault.maxDeposit(address(this));
            uint toDeposit = maxDeposit < balance ? maxDeposit : balance;
            token.approve(address(vault), toDeposit);
            vault.deposit(toDeposit, address(this)); // <== As seen here
        }
    }
    // If no vault is found, do nothing
}

```

The issue arises when the token itself is a vault for another ERC20 token. The router lacks awareness of nested vaults and will misbehave in such cases.

Proof of Concept: Consider the following setup:.

- USDC (base token).
- Yearn USDC Vault (a Yearn vault that accepts USDC and returns yvUSDC shares).
- Yearn yvUSDC Vault (a hypothetical meta-vault that accepts yvUSDC tokens as assets and returns yvyvUSDC shares).

Now, assume the router is configured with the following token-to-vault pairs:.

- USDC → Yearn USDC Vault.
- yvUSDC → Yearn yvUSDC Vault.

At this point, the Yearn USDC Vault yields yvUSDC shares, and the Yearn yvUSDC Vault yields yvyvUSDC shares.

Since yvUSDC exists both as an asset (for the meta-vault) and as a share (from the first vault), this dual role causes issues. When withdrawing or switching vaults for the yvUSDC token, the router aggregates both balances, potentially preventing it from fulfilling USDC withdrawals because the associated shares were already deposited or withdrawn.

Recommendation: Consider documenting this limitation so users are aware that nested vaults are not supported.

Mangrove: Acknowledged. Added docs in commit [b4a4188b](#).

Cantina Managed: Acknowledged.

3.3.2 ERC4626 vaults with hooks on deposits could reenter in the make offer flow

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: During the ERC4626Kand1 create offers flow we will end up in the `__posthookSuccess__` call. This function attempts to push the BASE/QUOTE tokens to the router, which then deposits them into the respective ERC4626 vault associated with each token:

```
function __posthookSuccess__(MgvLib.SingleOrder calldata order, bytes32 makerData)
    internal
    override
    returns (bytes32 repostStatus)
{
    // Handle dual offer posting
    transportSuccessfulOrder(order);

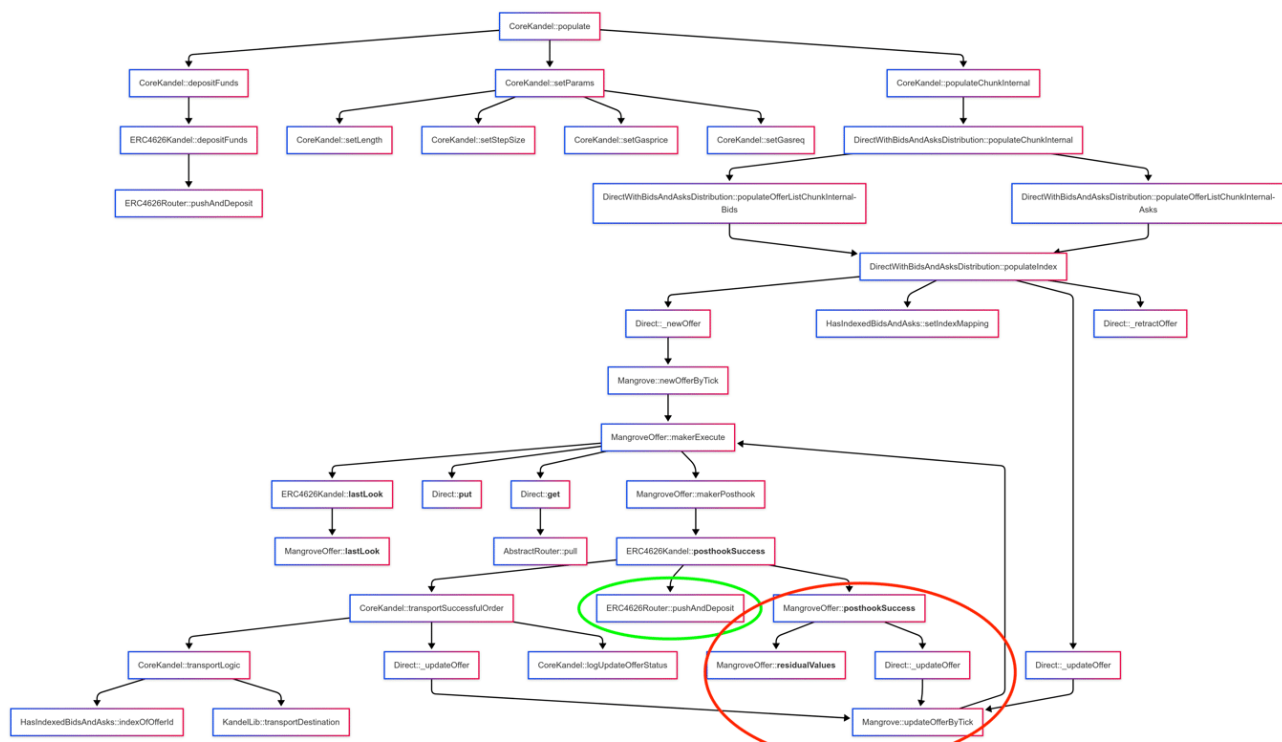
    // If first puller, the router should deposit liquidity into the vault
    uint baseBalance = BASE.balanceOf(address(this));
    uint quoteBalance = QUOTE.balanceOf(address(this));

    erc4626Router().pushAndDeposit(BASE, baseBalance, QUOTE, quoteBalance);
    // Reposting offer residuals if any - but do not call super, since Direct will flush tokens unnecessarily
    repostStatus = MangroveOffer.__posthookSuccess__(order, makerData);
}
```

If the ERC4626 vault itself has a deposit hook (such as Euler's Vaults, which support deposit hooks), there is a possibility that the call could reenter ERC4626Kand1 from the vault's hook.

Attached is a callgraph of the make offer flow with two highlights:

1. Green: where it reenters.
2. Red: affected components from state changes after reentrancy.



This issue is considered low risk for the following reason:

- The `ERC4626Router` is responsible for depositing into the vault, and most hook implementations allow the `msg.sender` to perform additional logic. If a hook is enabled in the vault, only the router could act maliciously. However, this is not a concern, as the router has no malicious intent.

Recommendation: Consider further analyzing whether this reentrancy could affect the offer-making flow, as time constraints prevented a definitive conclusion. If no changes are made, consider documenting this behavior.

Mangrove: Acknowledged.

Cantina Managed: Acknowledged.

3.3.3 Rounding up in ERC4626 withdrawal can cause the withdraw funds to fail

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: The `withdrawFundsForToken` function withdraws a specified amount of assets from the router, which in turn attempts to withdraw from both the local balance and the ERC4626 vault. An issue arises because ERC4626 rounds up shares slightly to avoid rounding errors. However, this can cause the call to revert if the router is unable to withdraw the exact amount specified as the withdraw will require slightly more shares than what we have available in the router:.

```
if (amount_ != 0) {
    erc4626Router().withdraw(token, amount_);
}
```

Due to time constraints, we were unable to fully validate all flows where this issue might occur.

Recommendation: Consider using `maxWithdrawal` to determine the maximum amount that can be withdrawn, and instead of attempting to withdraw a fixed amount, withdraw as much as possible. This also provides additional protection for the Morpho or other ERC4626 vaults implementation, as they might enforces various withdrawal caps.

Mangrove: Fixed in commit [e32c7236](#).

Cantina Managed: Fix verified.

3.4 Informational

3.4.1 Documentation and minor issues

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Throughout the contracts we can see various typos or minor issues that we aggregated into one issue:

- In `MorphoVaultRouter.sol#L6`: The `IMangrove` import is not used.
- In `MorphoKandelSeeder.sol#L1`: There's a missing `SPDX-License-Identifier` at the top of the file.
- In `ERC4626KandelSeeder.sol#L62`: The `NewERC4626Kandel` params are wrongfully emitted, because the `owner` and `address(kandel)` parameters are inverted in the current version. It should be:

```
emit NewERC4626Kandel(  
    owner, olKeyBaseQuote.hash(), olKeyBaseQuote.flipped().hash(), address(kandel), address(kandel)  
);
```

- In `ERC4626Kandel.sol#107-109`: The `makerData` is not returned in the `__lastLook__`, this function just calls the parent function. So it'd be better if you just remove it.
- In `ERC4626Kandel.sol#L44`: `IOfferLogic` and `AbstractRouter` imports aren't used.
- In `ERC4626Router.sol#L87`: There's an unnecessary condition `maxRedeemable < shares`, because this is already checked on L86 and the `maxRedeemable` cannot be lower than `shares`.
- In `ERC4626Router.sol#L76`: The `setVaultForToken` function has some weird behaviors that might not be fully understood:
 1. If a new vault is set to `address(0)`, the whole balance from the old vault remains in the router.
 2. If the new vault's maximum deposit is lower than before, then part of the past balance remains in the router as well.

This should be properly documented.

- In `ERC4626Router.sol#L134`: The docstrings for the `pushedAmount` return value should clarify that if the token handled takes fees on transfer, the returned amount might be larger than the actual amount pushed.
- In `ERC4626Router.sol#L97`: The docstrings for the `tokenBalanceOf` function should state that the balance returned accounts for tokens in the contract plus the ones in the corresponding vault. Also, should specify whether potential fees are already accounted or not in the returned amount.
- The docstrings for the `MorphoVaultRouter` contract are duplicated.

Recommendation: Consider fixing the minor issues mentioned above.

Mangrove: Acknowledged.

Cantina Managed: Acknowledged.

3.4.2 Missing events

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Some functions throughout the codebase may benefit from emitting events to log sensitive actions and changes:

- `ERC4626Router.sol#L62`: `adminWithdrawTokens` and `adminWithdrawNative`.
- `ERC4626Router.sol#L92`: `setVaultForToken`. Note that in `MorphoVaultRouter` there's an `event VaultSet` declared, although it's never used.

Recommendation: Consider emitting relevant events in the functions mentioned above.

Mangrove: Fixed in commit `d4beedc5`.

Cantina Managed: Fix verified.

3.4.3 Missing forceApprove on ERC4626Router deposit

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: The ERC4626Router deposits assets into the vault using the `_deposit` function. In this function, an approval is performed beforehand. If, for any reason, the vault does not consume the full approval and the token is one like USDT, which reverts if there are any previous approvals, the entire deposit will revert. While the chances of this happening are very low, as a safeguard, it is better to use OpenZeppelin's `forceApprove`, which resets approvals before setting them to the new value.

```
function _deposit(ERC20 token) internal {
    IERC4626 vault = vaults[token];
    if (address(vault) != address(0)) {
        uint balance = token.balanceOf(address(this));
        if (balance > 0) {
            uint maxDeposit = vault.maxDeposit(address(this));
            uint toDeposit = maxDeposit < balance ? maxDeposit : balance;
            token.approve(address(vault), toDeposit);
            vault.deposit(toDeposit, address(this));
        }
    }
    // if no vault is found, do nothing
}
```

Recommendation: Consider using OpenZeppelin's `forceApprove` instead of `approve`.

Mangrove: Fixed in commit [7604704c](#).

Cantina Managed: Fix verified.

3.4.4 Lack of validation for supported asset in setVaultForToken function

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: The `setVaultForToken` function of the ERC4626Router contract does not check whether the passed `token` is indeed the asset supported by the passed `vault` contract. Although only a trusted role in the system is expected to call this function, adding a validation should help avoid unexpected errors when setting tokens for vaults.

Recommendation: Consider including a `require` statement to check that `address(token) == vault.asset()` in the `setVaultForToken` function.

Mangrove: Fixed in commit [245eccfc](#).

Cantina Managed: Fix verified.

3.4.5 Low-level calls should use abi.encodeCall to create calldata

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: All low-level function calls in the `MangroveERC4626KandelVault` contract use Solidity's built-in `abi.encodeWithSignature` function, which does not perform full type checks of the parameters.

Recommendation: To avoid potential errors when creating the calldata for low-level calls, it's advisable to rely on `abi.encodeCall` instead, which performs full type checks. Also consider documenting why low-level calls are needed in this contract, instead of using regular high-level calls.

Mangrove: Fixed in commit [a86830f1](#) by adding interfaces.

Cantina Managed: Fix verified.

3.4.6 Various Kandel functions won't be able to be called via the MangroveERC4626KandelVault contract

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: When a MangroveERC4626KandelVault is deployed, the vault itself becomes the admin of the Kandel. This means that only the vault can call onlyAdmin functions. The following functions are restricted to onlyAdmin in the Kandel:

- adminWithdrawTokens.
- adminWithdrawNative.
- setVaultForToken.
- setBaseQuoteTickOffset.
- populateFromOffset.
- populateChunkFromOffset.
- setStepSize.
- setGasprice.
- setGasreq.
- populate.
- populateChunk.
- withdrawFunds.
- retractAndWithdraw.
- retractOffers.
- approve.
- withdrawFromMangrove.
- setAdmin.

However, only the following functions are actually called within the vault or can be called by the vault:

- setVaultForToken.
- adminWithdrawTokens.
- adminWithdrawNative.
- withdrawFunds.
- withdrawFromMangrove.
- setBaseQuoteTickOffset.
- populate.
- populateChunk.
- retractOffers.
- retractAndWithdraw.

Recommendation: Consider double-checking whether any additional functions should be callable by the MangroveERC4626KandelVault.

Mangrove: Acknowledged.

Cantina Managed: Acknowledged.